COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	07/22/2026	**DATE SUBMITTED:**	07/22/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 3.1 Creating a Singly Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

#include <iostream>

template <typename T>
class SingleList{
    public:
        T data; //this contains the data to be stored in the node
        SingleList<T>* next = nullptr;
};

template <typename T>
void ListTraversal(SingleList<T>* head){

    if (head == nullptr){
        std::cout << "The list is empty";
        return;
    }

    while(head != nullptr){
        // PRINT data OF n
        std::cout<<head->data;

        if(head->next != nullptr){
            std::cout << "->";
        }

        // GO TO NEXT NODE n -> next
        head = head->next;
    }

    std::cout << std::endl;
}

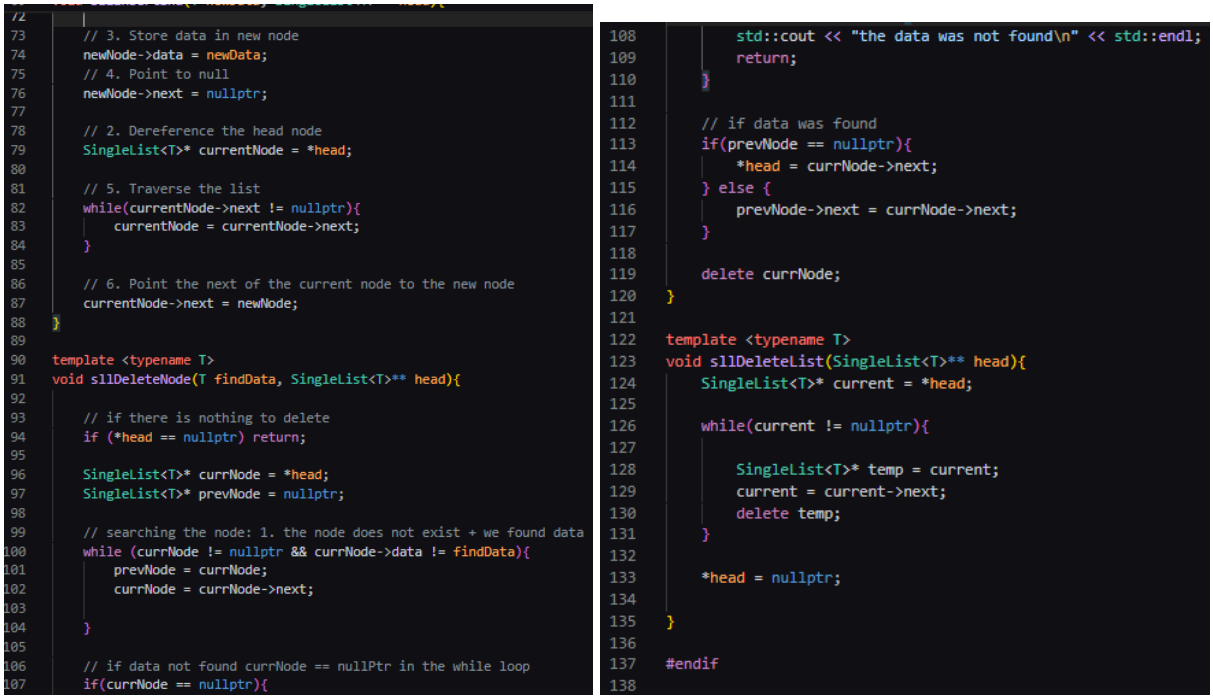
template <typename T>
void sllinsertHead(T newData, SingleList<T>** head){
```

```cpp
    // 1. Allocate new memory for the new code
    SingleList<T>* newNode = new SingleList<T>;
    // 2. Put our data into the new node
    newNode->data = newData;
    // 3. Set next to the new node to point to the previous head
    newNode->next = *head;
    // 4. Reset head to the new node
    *head = newNode;
}

template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode){

    //1. Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        //2. If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //3. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //4. Store data in the new node
    newNode->data = newData;
    //5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    //6. Point previous node to the new node
    prevNode->next = newNode;
}

template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    // 1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
```



```
72
73      // 3. Store data in new node
74      newNode->data = newData;
75      // 4. Point to null
76      newNode->next = nullptr;
77
78      // 2. Dereference the head node
79      SingleList<T>* currentNode = *head;
80
81      // 5. Traverse the list
82      while(currentNode->next != nullptr){
83          currentNode = currentNode->next;
84      }
85
86      // 6. Point the next of the current node to the new node
87      currentNode->next = newNode;
88  }
89
90  template <typename T>
91  void sllDeleteNode(T findData, SingleList<T>** head){
92
93      // if there is nothing to delete
94      if (*head == nullptr) return;
95
96      SingleList<T>* currNode = *head;
97      SingleList<T>* prevNode = nullptr;
98
99      // searching the node: 1. the node does not exist + we found data
100     while (currNode != nullptr && currNode->data != findData){
101         prevNode = currNode;
102         currNode = currNode->next;
103
104     }
105
106     // if data not found currNode == nullPtr in the while loop
107     if(currNode == nullptr){
```

```
108         std::cout << "the data was not found\n" << std::endl;
109         return;
110     }
111
112     // if data was found
113     if(prevNode == nullptr){
114         *head = currNode->next;
115     } else {
116         prevNode->next = currNode->next;
117     }
118
119     delete currNode;
120 }
121
122 template <typename T>
123 void sllDeleteList(SingleList<T>** head){
124     SingleList<T>* current = *head;
125
126     while(current != nullptr){
127
128         SingleList<T>* temp = current;
129         current = current->next;
130         delete temp;
131     }
132
133     *head = nullptr;
134
135 }
136
137 #endif
138
```

Figure 1. Screenshot of the Header File singly_ll.h.

This figure contains the screenshot of the code in the header file named singly_ll.h, which contains the standard #ifndef, #define, and #endif statements used in standard C based header files. An include command is added to the header files to provide the necessary dependencies when using std commands like cout and endl. Starting with the template and typenames, these parts serve as the basis for the classes and functions to be used as basic building blocks of processes that other cpp files can use. The use of these header files is to provide a reusable code and a basic form of a library that can clean up the code of the main cpp files used for compilation. Compared to cpp files, these header files do not compile on their own but rather, they simply link to the cpp file to complete a certain instruction or block of code when these files are compiled into one binary. The functions contained in this file are ListTraversal, sllinsertHead, sllGeneralInsert, sllinsertEnd, sllDeleteNode, and sllDeleteList, which are used for displaying, manipulating, and adjusting characters in sequence. A singly linked list starts with a head and then a node, which could then be joined together by other nodes to allocate their own set of memory spaces. Typenames behave as placeholders for data types to allow this code template to be reused in other programs.

```
clude <iostream>
clude "singly_ll.h"

main(){
// creating a node stored in the stack
SingleList<char>* head = nullptr;
SingleList<char>* second = nullptr;
SingleList<char>* third = nullptr;
SingleList<char>* fourth = nullptr;

// allocates memory in the heap
head = new SingleList<char>;
second = new SingleList<char>;
third = new SingleList<char>;
fourth = new SingleList<char>;

// storing a value inside the node
head->data = 'J';
second->data = 'P';
third->data = 'C';
fourth->data = 'L';

// linking the nodes
head->next = second;
second->next = third;
third->next = fourth;
fourth->next = nullptr;

std::cout << head->next << std::endl;
std::cout << second->next << std::endl;
std::cout << third->next << std::endl;
std::cout << fourth->next << std::endl;

std::cout << "Testing of traversal\n";
ListTraversal(head);
```

```
std::cout << "Testing of sllinsertHead\n";
sllinsertHead('Q', &head);
ListTraversal(head);

std::cout << "Testing of sllGeneralInsert\n";
sllGeneralInsert('X', head->next->next);
ListTraversal(head);

std::cout << "Testing of sllinsertEnd\n";
sllinsertEnd('U', &head);
ListTraversal(head);

std::cout << "Testing of deleting the node\n";
sllDeleteNode('U', &head);
ListTraversal(head);

std::cout << "Deleting all nodes\n";
sllDeleteList(&head);
ListTraversal(head);

return 0;
```

Figure 2. Screenshot of the Main Program File.

This figure contains the screenshot of the main cpp file, which has the instructions used to manipulate the characters in a singly linked list. The set of four characters stored in their respective nodes are J, P, C, and L to be used in testing the traversal, storing values, linking and deleting nodes, and various insertion methods. This main program demonstrates the creation and allocation of nodes inside memory through the declaration of SingleList with a char data type pointing to null as the starting point of each variable. Because nodes are not treated in the same way as lists, wherein individual elements can be reassigned at any index, nodes have to be sequentially allocated, assigned, linked, and deleted. After declaration and allocation, four of the characters are assigned to the data space of head, second, third, and fourth. Then, these values are linked next to each node to allow for functions to insert other values or characters into the sequence of nodes, and modify and delete one of the nodes in the linked lists.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.